



ONCHAIN CORE BANKING

TECHNICAL OVERVIEW · V1.0 · SEPTEMBER 2026

On-Premise Deployment

A complete picture of the DFNS platform when it runs inside your own cloud account: the architecture and its layers, every component you operate, the signing options, where each kind of key lives, how a deployment proceeds from an empty account to a signed transaction, and the operating model afterwards. Written for infrastructure, SRE, and security teams evaluating or preparing a deployment.

COVERS

ARCHITECTURE

SIGNING TIERS

KEY CUSTODY

DEPLOYMENT FLOW

INTEGRATIONS

DAY-2 OPERATIONS

SCOPE & LIMITS

INTRODUCTION

About this document

DFNS is an Onchain Core Banking platform. It gives an organization the services needed to create and operate blockchain accounts and to sign transactions under policy: authentication and user lifecycle, wallets across many chains, a signing tier that holds the cryptographic keys, an approval and compliance engine, blockchain integration and indexing, and operator dashboards.

This document describes the **full on-premise** deployment model, in which that entire platform runs inside an AWS account you own and operate. It explains what exists, how the pieces fit together, and what running it involves. Step-by-step procedures live in the deployment handbook that ships with each release; this overview tells you the shape of the system so you can evaluate it and plan for it.

Audience. Infrastructure, SRE, and security engineers. Familiarity with Kubernetes, Terraform, DNS, and PKI is assumed. The document explains the platform's behaviour, not those tools. No prior knowledge of DFNS is required.

What is inside

01 **Where the on-premise model sits** deployment shapes, and where full on-premise fits

02 **Architecture: four layers** the sequenced infrastructure layers

03 **The moving pieces** application services, shared infrastructure, signing tier

04 **Trust and key material** the four trust domains and where keys live

05 **What DFNS delivers** the signed bundle and image distribution

06 **The deployment flow** the attended, gated procedure end to end

07 **What you provide** prerequisites and the day-0 decisions

08 **Integration capabilities** email, identity, chains, observability, more

09 **Day-2 operations** upgrades, rotation, redundancy, diagnostics

10 **Scope boundaries and current limits** what your architecture review can rely on

11 **Companion guides** the detailed material referenced throughout

SECTION 01

Where the on-premise model sits

DFNS supports a spectrum of deployment shapes that differ in who runs what. The signing infrastructure (the tier that holds keys and produces signatures) and the platform (APIs and dashboards) can each sit with DFNS or with you.

SHAPE	SIGNING INFRASTRUCTURE	PLATFORM (APIS, DASHBOARDS)
DFNS-hosted (SaaS)	DFNS	DFNS
Hybrid MPC, shared quorum	Split: you run some MPC signers, DFNS runs the rest	DFNS
Hybrid MPC, full quorum	You run all MPC signers	DFNS
Client-hosted HSM	Your HSM plus a driver you operate; DFNS runs the proxy	DFNS
Full on-premise THIS MODEL	You (MPC in-cluster, or your HSM)	You

In the full on-premise model, the entire platform runs in an AWS account you own and operate: API services, dashboards, databases, message bus, secret store, and the signing tier. The hybrid shapes above are covered by their own guides; this document covers the full on-premise model, including both of its signing options (MPC and HSM), because the signing tier is a choice *inside* the model.

RUNTIME INDEPENDENCE

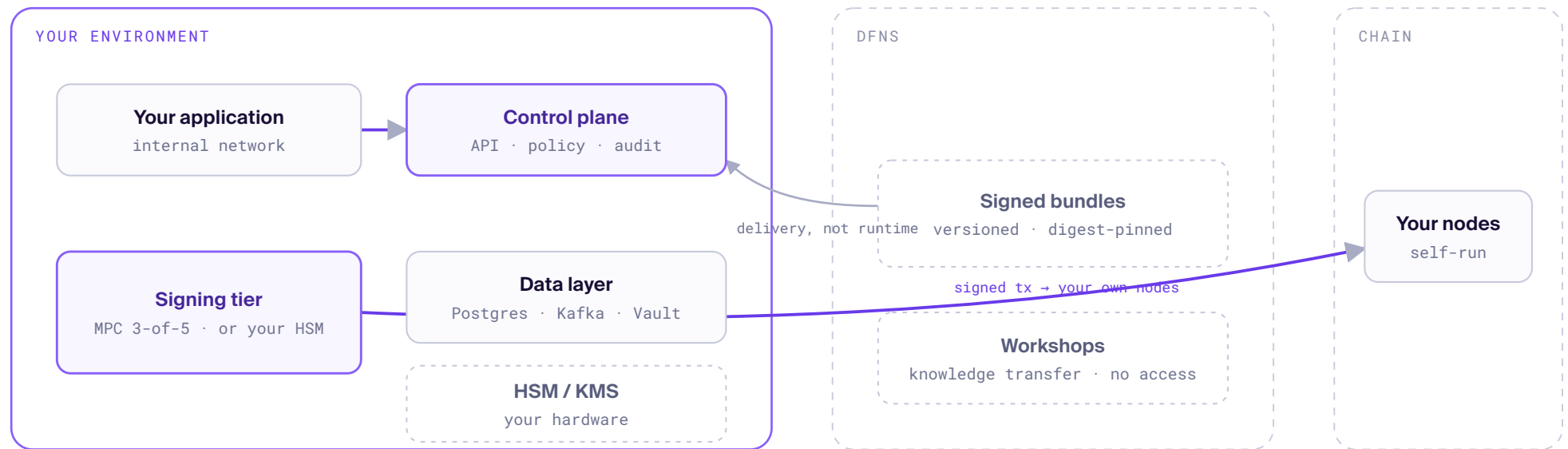
After delivery, the environment runs with **no runtime dependency on DFNS infrastructure**. It operates with no connectivity to DFNS, and container images can be imported offline into your own registry.

DIAGRAM · SECTION 01

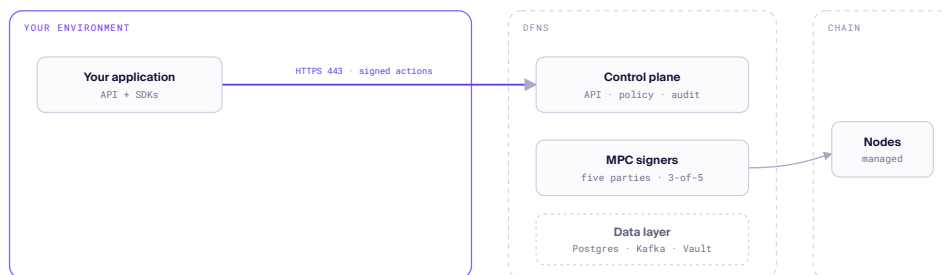
Deployment shapes, drawn

In the full on-premise model (top), every runtime component sits inside your walls; DFNS ships signed bundles and knowledge, never traffic. The hosted shapes below are shown for contrast — they differ only in where the signing tier and platform run.

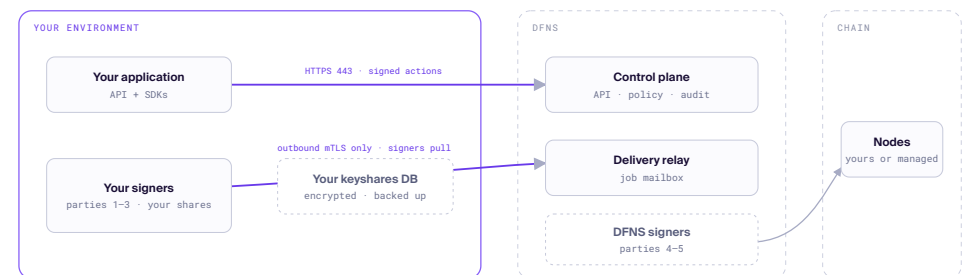
FULL ON-PREMISE this document's model — zero runtime connection to DFNS



DFNS-HOSTED (SAAS) for contrast



HYBRID MPC for contrast



YOUR ENVIRONMENT KEY CUSTODY PATH
 DFNS OPTIONAL / DELIVERY-TIME
 CHAIN RUNTIME COMPONENT

SECTION 02

Architecture: four layers

The deployment is organized as four sequenced infrastructure layers. Each layer is a separate Terraform root with its own state; later layers read the outputs of earlier ones, so the order is fixed and the tooling enforces it. Everything above the first layer is standard Kubernetes: the application is delivered as Helm charts and container images, deployed onto the cluster the first layer creates.

LAYER	WHAT IT PROVISIONS
L1 · Substrate	VPC and networking, the Kubernetes (EKS) cluster, a managed PostgreSQL cluster (Aurora Serverless v2), managed Kafka (MSK), managed Redis (ElastiCache), a Route53 DNS hosted zone with ACM TLS certificates, an SSM bastion, and an optional CloudFront CDN with web application firewall (WAF) at the edge.
L2 · Platform	The Istio service mesh (strict mutual TLS) and ingress gateway, the secret store (either Vault in raft high-availability with AWS KMS auto-unseal, or the native AWS secrets backend), namespaces and role-based access control, storage classes, the registry pull secret, and optional node autoscaling.
L3 · Product	Per-service databases on the shared PostgreSQL cluster, Kafka topics and access controls, secret-store seeding, and the platform Helm umbrella release: all application services, initialization jobs, and dashboards.
L4 · Signing	Your chosen signing tier: an in-cluster MPC signer cluster, or the HSM proxy and driver pair in front of your HSM, plus an optional governance engine.

The layer sequence is the deployment path. The tooling runs unattended within a layer but stops at defined pause points where only you can act (see Section 06). Reading the layers top to bottom is also the fastest way to understand the system: substrate first, then the platform fabric, then the application, then the keys.

SECTION 03 · PART 1 OF 2

The moving pieces

Application services

The product layer deploys one Helm umbrella release containing roughly twenty workloads:

- ◆ **Twelve core API services.** Authentication and organization or user lifecycle (based on WebAuthn passkeys), wallets (lifecycle, transfers, broadcast), signer coordination (driving the signing tier), policies (approval and compliance rules), blockchain integrations (multi-chain transaction construction, fees, network metadata), blockchain indexing (block ingest and transaction finalization), admin, webhooks, aliases, market data, and metrics. Several services run companion cron or indexer workloads from the same image.
- ◆ **Two initialization jobs.** Database schema migrations, and a one-shot platform bootstrap that generates the issuer key and the initial staff organization.
- ◆ **Two dashboards.** The end-user dashboard and the staff or operator dashboard.

All application images are built on distroless Node.js, run as non-root, and are published for arm64 (Graviton-class nodes). They share a single unified platform version: every service in a release carries the same version number, and images are pinned by digest.

Shared infrastructure

- ◆ **PostgreSQL.** Nine logical databases, one per service domain (auth, permissions, wallets, keystores, policies, blockchain integrations, aliases, markets, events).
- ◆ **Kafka.** The platform's event bus. Topics and per-service credentials are provisioned in the product layer; indexers and cron workloads are Kafka consumers.
- ◆ **Redis.** The caching tier.
- ◆ **Secret store.** Either Vault or the native AWS secrets backend, chosen on day 0 (see Section 07). It holds service secrets, per-service transit or KMS keys, and, with Vault, the PKI mounts used by the signing tier.
- ◆ **Istio.** In-cluster mutual TLS between all services (strict mode) and the ingress gateway, fronted by an AWS network load balancer (NLB).

SECTION 03 · PART 2 OF 2

Signing tier options

The signing tier is the component that holds cryptographic keys and produces signatures. The on-premise model offers two options, chosen on day 0, plus an optional governance component.

MPC, in-cluster

Five signer parties with a 3-of-5 signing threshold (other topologies are supported by arrangement), plus a delivery relay that carries protocol rounds between the parties. Key shares are generated inside the signers at provisioning through distributed key generation, so no complete private key ever exists in one place. Each party's certificates are issued from a dedicated PKI mount in your secret store.

HSM

An in-cluster `hsm-proxy` and an `hsm-driver` that runs next to your HSM, typically on separate amd64 hosts, possibly outside the cluster and even outside the VPC. The driver dials *out* to the proxy over mutual TLS, so no inbound connectivity to the HSM side is ever required. Supported HSM platforms include Securosys Primus (on-premise appliance or CloudsHSM service), Thales Luna, AWS CloudHSM, and IBM EP11 or HPCS-class devices. Wallet private keys are generated inside the HSM and stored only as wrapped blobs (AES-GCM under a non-extractable root wrap key that never leaves the HSM) in a PostgreSQL keystore database you operate. Both ECDSA secp256k1 and Ed25519 signing are supported.

Governance engine, optional

An additional approval and attestation component that signs governance decisions with its own key, which is HSM-wrapped when the HSM tier is present.

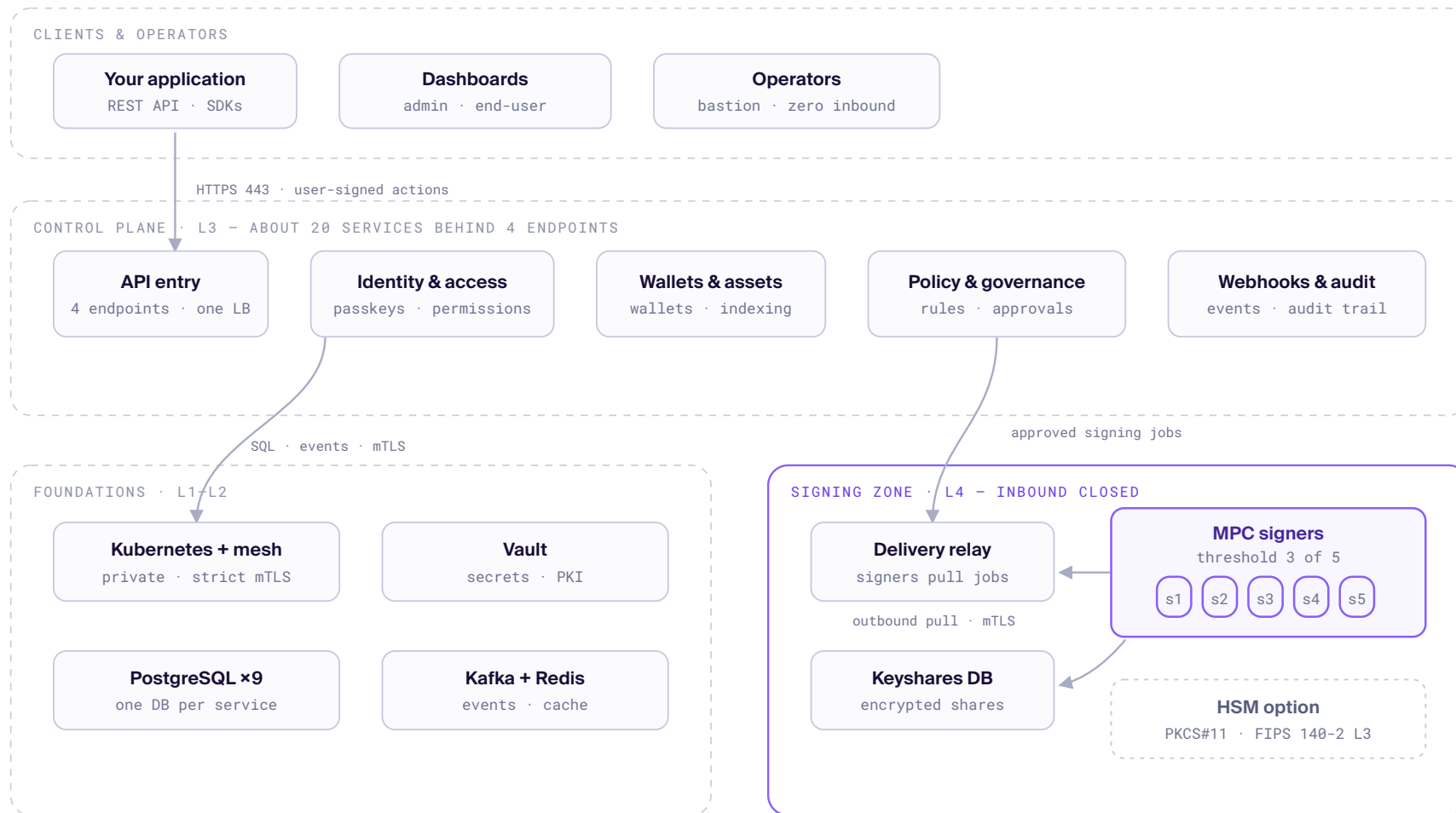
CUSTODY

Wallet signing keys are **never held by the platform API services and never held by DFNS**. With MPC they are split into shares across the signer parties; with an HSM they stay inside your HSM, and only wrapped blobs ever reach the keystore database.

DIAGRAM · SECTIONS 03-04

The moving pieces, on one canvas

Every runtime component from sections 03 and 04, in its zone. All four zones run inside your environment; the violet border marks the signing zone, which accepts no inbound connections — signers pull their jobs from the delivery relay over mTLS.



 KEY MATERIAL LIVES HERE OPTIONAL / ALTERNATIVE TIER PLATFORM COMPONENT

SECTION 04

Trust and key material

The platform's cryptography is organized into four independent trust domains. Knowing where each kind of key lives is most of the security review.

TRUST DOMAIN	MATERIAL	WHERE IT LIVES
Wallet signing keys	ECDSA secp256k1 / Ed25519	MPC: split into key shares across the signer parties. HSM: inside your HSM, with only wrapped blobs in the keystore database. Never held by the platform APIs, never held by DFNS.
Platform authentication	Ed25519 token-signing (issuer) key	Your secret store (Vault KV or AWS Secrets Manager), envelope-encrypted under your KMS. Readable only by the auth service.
Signing-tier mutual TLS	Per-environment private CA with intermediates; ECDSA P-256 leaf certificates per component	CA and intermediates in your secret store; leaf certificates mounted into the signing-tier pods. Leaf certificate common names are load-bearing: they encode a component's identity and are enforced at the TLS handshake.
Service mesh	Istio workload certificates (short-lived)	In-cluster CA; rotated automatically.

Two custody facts deserve emphasis. First, whoever holds the **Vault recovery keys and root token** (taken at initialization, Section 06) controls the secret store. Custody of those values is yours, established during deployment, and the root token is revoked after day 0. Second, on the HSM path the **root wrap key together with the keystore database** are jointly sufficient, and jointly necessary, to recover every wallet key. Both must be backed up: HSM-native partition backup or replication for the key, and standard PostgreSQL backup for the database. Losing the root wrap key with no HSM backup makes every wrapped key permanently unrecoverable.

Customer-held encrypted backup (MPC)

For MPC deployments, an optional customer-held encrypted backup tier is available. Every key share is additionally encrypted to a public key you generate and hold offline, then written to a versioned S3 bucket. Recovery from these backups is fully independent of DFNS.

SECTION 05

What DFNS delivers

A deployment is delivered as a single signed, versioned bundle. Its contents:

- ◆ **Handbook and runbooks.** An offline-readable deployment handbook, plus per-topic runbooks: day-0 walkthrough, connectivity, DNS delegation, Vault initialization, third-party secrets, HSM ceremony, organization bootstrap, day-2 operations, diagnostics, and sizing.
- ◆ **Terraform roots and modules.** The four layer roots (Section 02) with all module dependencies vendored into the bundle, plus a Makefile that enforces preflight checks and layer ordering. Third-party providers can be mirrored into the bundle for restricted-egress environments.
- ◆ **Helm charts.** The platform umbrella and the signing-tier charts (signers, HSM, HSM driver, governance, initialization), all vendored.
- ◆ **Values templates.** An annotated `terraform.tfvars.example` and a customer values seed. These two files are the only ones you edit.
- ◆ **Image manifest.** Every container image with its registry path, tag, and sha256 digest, stamped at release time.
- ◆ **Integrity material.** A signed checksum list covering every file in the bundle, the release public key to verify it, and a cosign public key to verify each container image signature.

Image distribution

Image distribution supports three channels, chosen per environment: credentials to a DFNS-provided registry (pull-through), replication into your own artifact repository, or fully offline OCI tarballs that you import into your own registry with a provided script. The offline channel is the recommended pattern for air-gapped production environments.

Upgrades arrive the same way. Each release is a complete new bundle rather than an in-place diff. You verify it before unpacking, then apply it layer by layer.

SECTION 06 · PART 1 OF 2

The deployment flow

Deployment is an attended, gated procedure. The tooling runs unattended within a layer but stops at defined **PAUSE** points where only you can act. A first deployment completes in one to two working days once prerequisites are ready.

01 Preflight

A provided check validates credentials, region service support, quotas, registry access, tool versions, and the domain before anything is created.

02 L1 apply

Substrate creation, the longest single step; the managed Kafka cluster dominates wall-clock time. Terraform prints four nameserver values.

03 DNS delegation **PAUSE 1**

You delegate the deployment domain to those nameservers at your registrar or parent zone. Certificate validation, and therefore everything TLS-facing, gates on this.

04 L2 apply

Service mesh, ingress, and the secret store.

05 Connectivity **PAUSE 2**

Confirm your path to the private cluster endpoint. The provided SSM bastion tunnel is the default; your own VPN or an in-VPC runner work equally.

06 Vault initialization and key custody **PAUSE 2B**

You run `vault operator init` and immediately take custody of the recovery keys and the initial root token, per your key-custody plan. The root token is used once more (Pause 3) and then revoked.

07 L3 stage 1

Databases, topics, and secret seeding.

SECTION 06 · PART 2 OF 2

The deployment flow, continued

08

Third-party secrets

PAUSE 3

You populate the secret store with the credentials only you can provide: chain RPC endpoints, email provider credentials, OIDC client configuration, and any optional integrations (Section 08). The per-engagement list ships with the bundle, and the next stage refuses to plan until required entries exist.

09

L3 stage 2

The application deploys: schema migrations, the platform bootstrap, then the umbrella release. Verification: the API answers on `api.<your-domain>`, both dashboards load, and the first staff administrator registers a passkey.

10

L4 apply

The signing tier. MPC is a single apply. The HSM path adds a **PAUSE 4 · CERTIFICATE CEREMONY**: you generate certificate signing requests with the provided script (private keys never leave your custody), DFNS signs them against your per-environment CA, and you load the signed certificates before the second apply. On the HSM path the first driver start also performs the **root-wrap-key ceremony**: the driver auto-provisions the root key on the HSM partition, and you back it up before any wallet exists.

11

Keystore registration and end-to-end proof

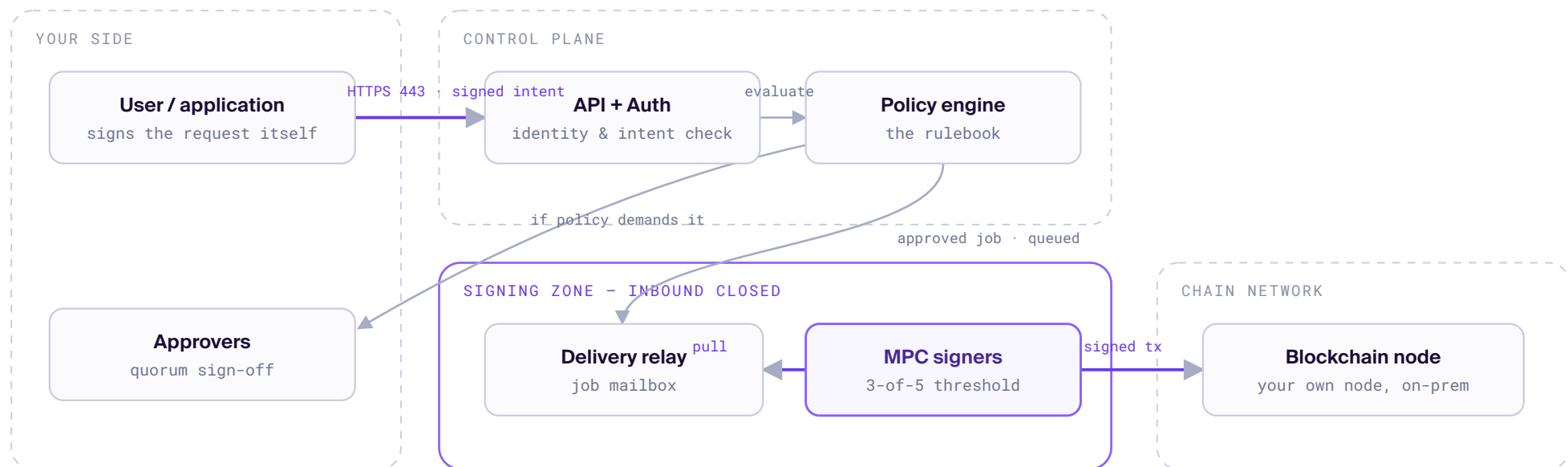
The signing cluster is registered as a key store through the staff dashboard, a WebAuthn-gated action. An organization is linked to it, and the deployment is proven by creating a wallet and signing a transaction end to end.

Proof of done. A deployment is considered complete only when a wallet has been created and a transaction signed end to end on the environment you now operate.

DIAGRAM · SECTION 06

One transaction, six control points

The end-to-end proof at the close of the deployment flow exercises this exact path. Every hop is inside your environment; no step can be skipped, and the signing zone only ever pulls — nothing dials into it.



- 01 User signs the intent 02 Auth verifies identity 03 Policy evaluates the rules 04 Approvers reach quorum 05 Threshold signature (3-of-5) 06 Your node broadcasts

 CUSTODY-CRITICAL HOP CONTROL POINT

SECTION 07

What you provide

- ◆ **A dedicated AWS account** in a region offering both EKS and MSK, with quota headroom. The reference deployment uses roughly 32 on-demand vCPUs across a small arm64 node group; the HSM path adds amd64 capacity for the driver-side components.
- ◆ **A domain** (or delegated subdomain) whose parent zone you control. The environment serves the apex and a wildcard, so dedicate it.
- ◆ **Per-layer deployer identities.** One distinct deployer identity per layer, plus read-only identities for human operators. Day 0 runs with broad rights; scoping down to least privilege is a documented hardening step after day 0.
- ◆ **A Terraform state backend.** One versioned, KMS-encrypted S3 bucket; the signing layer's state warrants a dedicated KMS key.
- ◆ **An operator workstation.** macOS or Linux with the standard toolchain (AWS CLI v2, OpenTofu or Terraform, kubectl, Helm, jq, dig, and the SSM session plugin). A bootstrap script installs everything, and the preflight check verifies it.
- ◆ **Secrets you own.** Vault custody values (Section 04), database master credentials kept in your secrets manager and never in files or state, registry credentials if using the pull-through channel, and the third-party credentials of Section 08.
- ◆ **HSM path only.** The provisioned HSM (partition, user, PIN), the vendor's PKCS#11 client configuration, hosts for the driver, a PostgreSQL instance for the keystore database, and, non-negotiable, a backup story for the root wrap key.

Seven day-0 decisions

Seven decisions shape the deployment and are captured on a decision sheet before anything is created: the signing profile (MPC, HSM, or governance), the compute model (static node groups or autoscaling), the domain scenario, the network path, the image registry channel, the optional CloudFront and WAF edge, and optional features (metrics, captcha).

One-way door. One decision cannot be reversed later: the secrets backend (Vault versus AWS-native) is fixed at day 0, with no cross-backend migration. Decide it deliberately.

SECTION 08

Integration capabilities

The platform connects to your surrounding systems through the integration points below. Email and at least one chain endpoint are effectively prerequisites; the rest are optional and provider-agnostic.

INTEGRATION	PURPOSE	NOTES
Email (SMTP or SendGrid)	Registration, recovery, and login emails; effectively an onboarding prerequisite	Works with your own relay over STARTTLS or TLS; see the dedicated on-premise email guide. A documented break-glass procedure exists for onboarding before email is live.
OIDC	Delegated login against your identity provider	OIDC-enforced organizations onboard without registration emails at all: the first identity-provider user with an admin-mapped claim becomes the organization admin.
Chain RPC endpoints	One per enabled blockchain network	Endpoints with embedded credentials are stored in the secret store, never in configuration files.
Market data / pricing APIs	Fee estimation, pricing, staking data	Optional; provider-agnostic configuration.
Webhooks	Outbound event delivery to your systems	Built-in service.
Observability	Structured JSON logs; OTLP trace export	Ship logs container-natively to your SIEM; point the OTLP exporter at your collector (Datadog or any OpenTelemetry-compatible backend). No bundled Prometheus or Grafana; bring your own.
Captcha (reCAPTCHA)	Login protection	Optional.
Slack	Operational notifications	Optional.
Container registry	Image supply	Your registry (offline import), a replication target, or DFNS-provided pull credentials (Section 05).

SECTION 09

Day-2 operations

Once the environment is live, running it is a small, well-defined set of routines. Everything below is designed so that the operating team owns it without a runtime dependency on DFNS.

- ◆ **Upgrades.** A new signed bundle per release. You verify it, replace the vendored modules and charts wholesale, then plan and apply layer by layer. Application-only version bumps are values changes.
- ◆ **Rollback.** Versioned Terraform state plus digest-pinned images. The runbooks cover both infrastructure and application rollback, and schema migrations ship with rollback scripts.
- ◆ **Certificate rotation.** Edge certificates renew automatically while DNS delegation holds. Signing-tier CA and leaf rotation is a documented secret-store procedure followed by a pod restart. Mesh certificates rotate themselves.
- ◆ **Secret-store hygiene.** After day 0, revoke the initial root token, move to scoped operator tokens and Kubernetes authentication, and enable audit logging.
- ◆ **Diagnostics.** A diagnostics runbook covers the common failure classes (initialization-job failures, database TLS trust, secret-store access, node-architecture mismatches), and all services emit correlatable structured logs.
- ◆ **HSM redundancy.** Production runs at least two drivers against an HSM pair that holds the same root wrap key through vendor-native replication. Each driver has its own leaf certificate, and they share one keystore database. The pull model load-shares without a load balancer.

Standing invariant, worth testing periodically: signing continues while any single driver or HSM endpoint is down. Exercising that failover on a schedule is the simplest way to keep confidence in the redundant configuration.

SECTION 10

Scope boundaries and current limits

Stated plainly, so your architecture review can rely on them.

- ◆ **Kafka and Istio are required.** Platform messaging is Kafka-only, and ingress plus in-cluster mutual TLS are Istio-native. Neither can be swapped out or deferred.
- ◆ **The Terraform layers are the deployment path.** Rendered Kubernetes manifests can be produced for inspection and security review, but a chart-only installation cannot provision databases, topics, secrets, or DNS, and pointing the kit at a hand-built cluster is not a supported path.
- ◆ **Architecture is mixed by design.** The application tier is arm64-only; HSM driver components are amd64. Plan node capacity for both when the HSM path is in scope.
- ◆ **The secrets backend is a one-way door** (Section 07).
- ◆ **TLS at the edge requires real DNS delegation** for certificate validation. An internal-only domain without edge TLS is the supported alternative.
- ◆ **Least-privilege access is a hardening step, not a starting point.** Day 0 runs with broad rights and scopes down afterwards.
- ◆ **Hierarchical-deterministic (HD) wallets are not currently available on the HSM signing path.**
- ◆ **Air-gapped or offline signing tiers, and HSM root-key rotation,** are engagement-scoped procedures rather than self-service features. Raise them with your DFNS solutions engineer during scoping.

SECTION 11

Companion guides

GUIDE	COVERS
DFNS Outbound Email On-Premise	Configuring the platform's outbound email against your own SMTP relay (full and Essentials editions).
User Onboarding Without Email	Break-glass onboarding by injecting a registration code directly in the database you operate.
Deployment handbook (in the bundle)	The authoritative step-by-step day-0 procedure, per-layer runbooks, and diagnostics.

For anything this overview leaves open, whether sizing, non-standard topologies, additional HSM vendors, or compliance questionnaires, contact your DFNS solutions engineer.